

Week 15: Approximation & policy-based RL

D. M. Pelt | Introduction to Reinforcement Learning



Universiteit
Leiden
The Netherlands

Overview

- Last weeks: planning and model-based RL
- This week: approximation and policy-based RL
- Explained for model-free RL, but applicable to model-based as well

Approximation

Curse of dimensionality

- Methods so far use a **table** of Q values
- In many problems, this table becomes **huge!**
 - E.g., state-space of chess is roughly 10^{47}

Curse of dimensionality

- Methods so far use a **table** of Q values
- In many problems, this table becomes **huge!**
 - E.g., state-space of chess is roughly 10^{47}



**Impossible
computationally**

**States/actions rarely
visited**

Storing values

- We can't directly store a value $v_{\pi}(s)$ for each state s

Storing values

- We can't directly store a value $v_{\pi}(s)$ for each state s
- Solution: approximate values using a function v with parameters w :

$$v_{\pi}(s) \approx v(s, w)$$

- Of course, number of parameters must be $\ll$ number of states

Prediction objective

- How do we ensure that $v(s, w)$ is a good approximation of $v(s)$?
- Solution: define an error measure:

$$VE(w) = \sum_s \mu(s) [v_\pi(s) - v(s, w)]^2$$

- $\mu(s)$ can be weighted with frequency of occurrence
- Policy evaluation becomes: find w that minimizes $VE(w)$

Gradient methods

- Find w that minimizes $VE(w)$... **How?**

Gradient methods

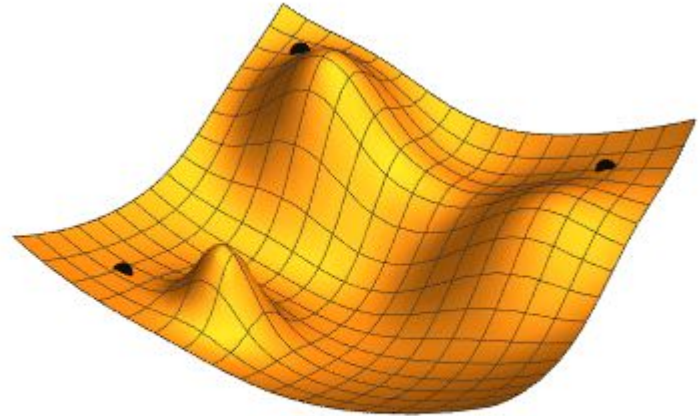
- Find w that minimizes $VE(w)$...
- Suppose we have a magic way of obtaining $v_{\pi}(s)$ for a given s
- Idea:
 - Initialize randomly
 - Loop over states, making $VE(w)$ slightly smaller each time

Gradient descent!

Gradient descent

Make $VE(w)$ slightly smaller each time:

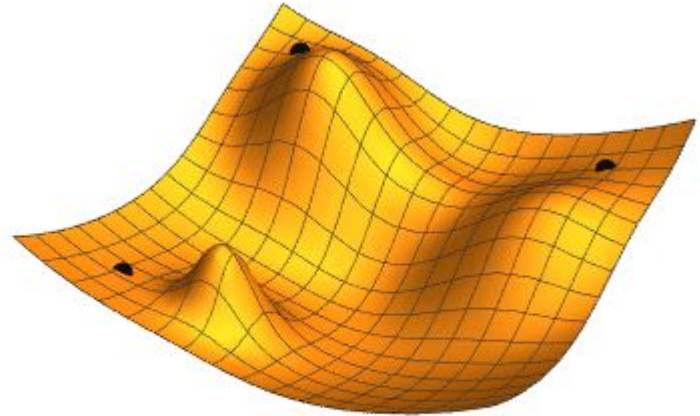
- Take derivative
- Make small step in direction of negative gradient



Gradient descent

Make $VE(w)$ slightly smaller each time:

- Take derivative
- Make small step in direction of negative gradient



Gradient descent

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \left[v_{\pi}(s) - \hat{v}(s, \mathbf{w}) \right]^2$$

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t - \frac{1}{2} \alpha \nabla \left[v_{\pi}(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right]^2$$

= ?

Make small step in direction of
negative gradient

Gradient descent

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \left[v_\pi(s) - \hat{v}(s, \mathbf{w}) \right]^2$$

$$\begin{aligned} \mathbf{w}_{t+1} &\doteq \mathbf{w}_t - \frac{1}{2} \alpha \nabla \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right]^2 \\ &= \mathbf{w}_t + \alpha \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t), \end{aligned}$$

Make small step in direction of
negative gradient

Gradient descent

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t),$$

Problem: we don't know $v_\pi(s)$...

Gradient descent

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t),$$

Problem: we don't know $v_\pi(s)$...

Solution: use methods we used before!

Monte Carlo, Temporal Difference, ...

Gradient Monte Carlo

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t),$$

Monte Carlo:

- Just generate episodes!
- Use Monte Carlo target (cumulative future discounted reward):

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[G_t - \hat{v}(S_t, \mathbf{w}) \right] \nabla \hat{v}(S_t, \mathbf{w})$$

Gradient Monte Carlo

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Algorithm parameter: step size $\alpha > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):

 Generate an episode $S_0, A_0, R_1, S_1, A_1, \dots, R_T, S_T$ using π

 Loop for each step of episode, $t = 0, 1, \dots, T - 1$:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

Gradient Monte Carlo

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

- Solution for storage of values

Does it also help with the problem of unvisited states?

Gradient Monte Carlo

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

- Solution for storage of values

Does it also help with the problem of unvisited states?

- Yes! Weights are shared between states:

Experience from observed states also updates policy for other states

- Approximation function should be chosen carefully

Gradient TD(0)

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t),$$

TD(0):

- Just generate episodes!
- Use single-step TD target:

Gradient TD(0)

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t),$$

TD(0):

- Just generate episodes!
- Use single-step TD target:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w}) \right] \nabla \hat{v}(S, \mathbf{w})$$

Gradient TD(0)

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Algorithm parameter: step size $\alpha > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose $A \sim \pi(\cdot | S)$

 Take action A , observe R, S'

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w})$

$S \leftarrow S'$

 until S is terminal

State aggregation

- Remaining question:

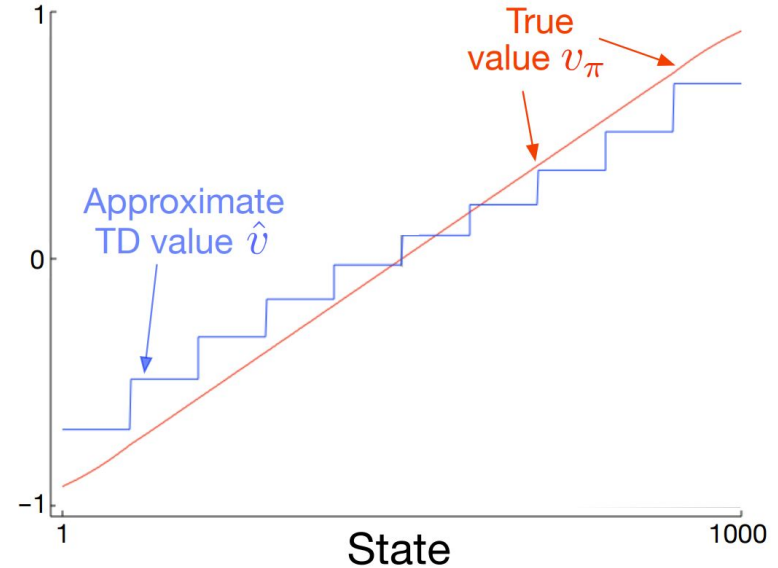
What to use for $v(s, w)$?

State aggregation

- Remaining question:

What to use for $v(s, w)$?

- Simple solution:
 - Group states, estimate one value for each group



Linear methods

- What to use for $v(s, w)$?
- Associate with each state s a **feature vector** x :

$$x(s) = (x_1(s), x_2(s), \dots, x_d(s))^T$$

- $x_i(s)$ is a **basis function**: many possibilities!

$$v(s, w) = \sum_{i=1}^d w_i x_i(s)$$

Linear methods

$$v(s, \mathbf{w}) = \sum_{i=1}^d w_i x_i(s)$$

- Derivative:

$$\nabla \hat{v}(s, \mathbf{w}) = \mathbf{x}(s)$$

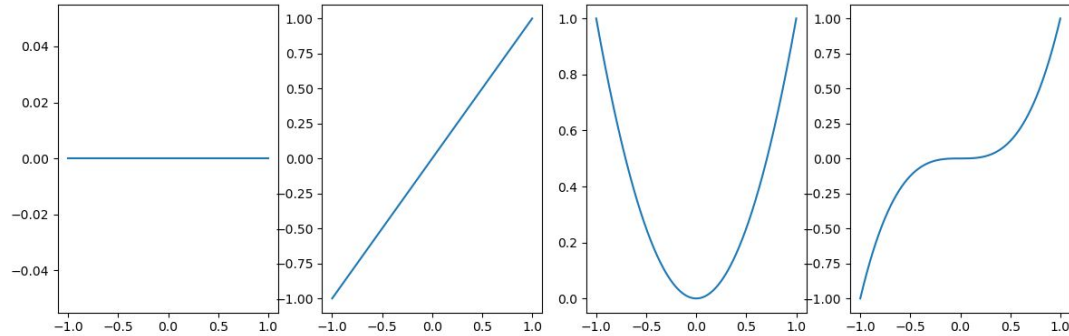
- MC update:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

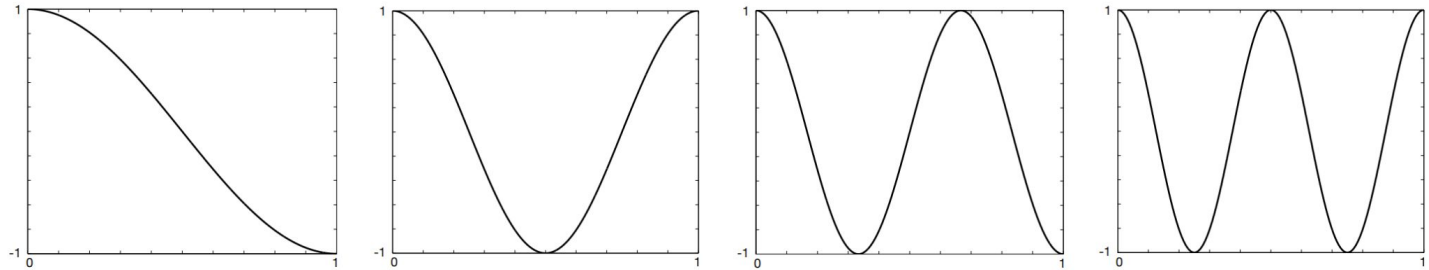
$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \mathbf{x}(s)$$

Feature construction

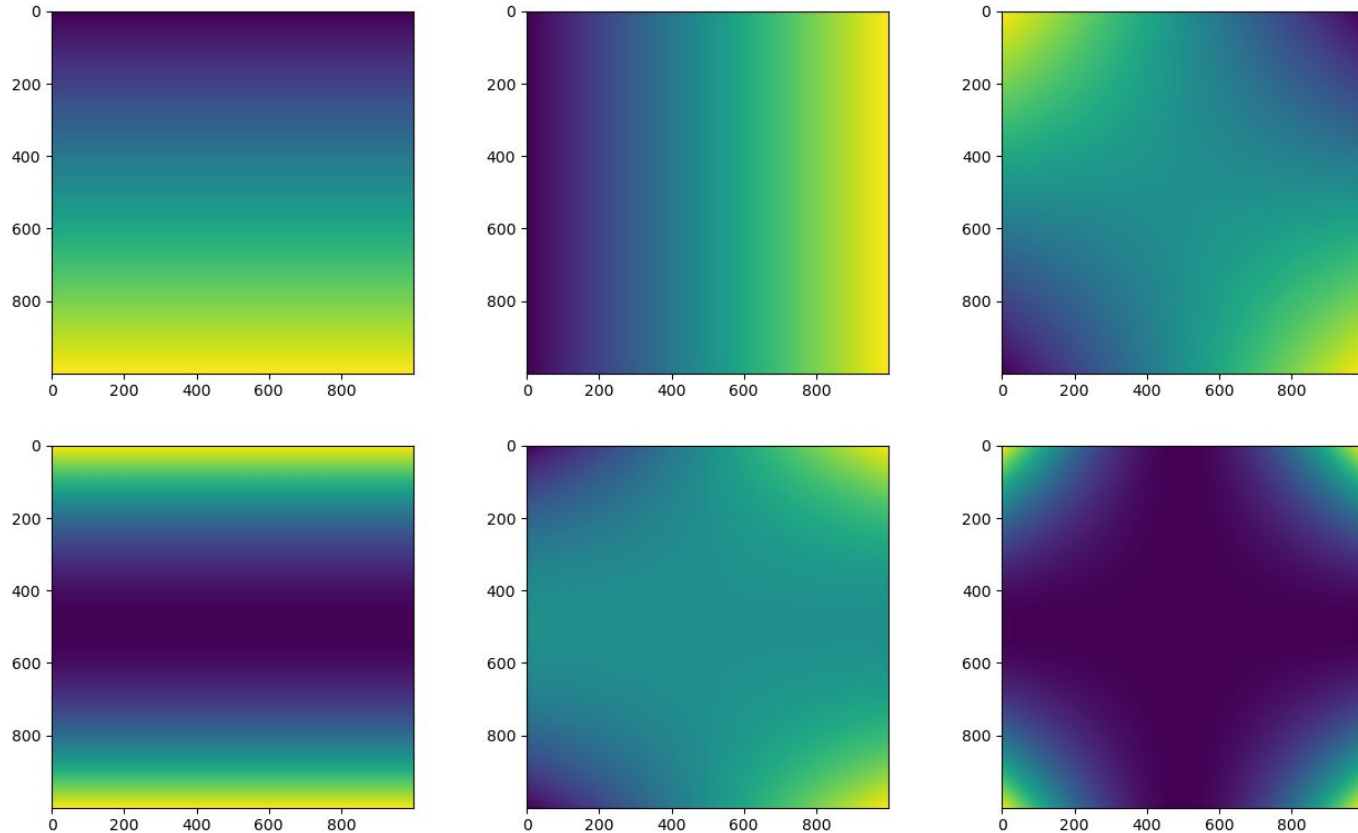
- Polynomial:



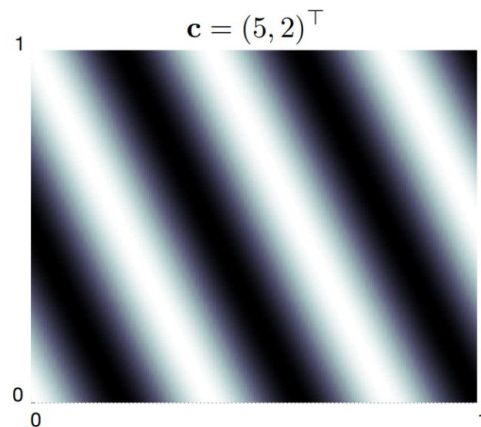
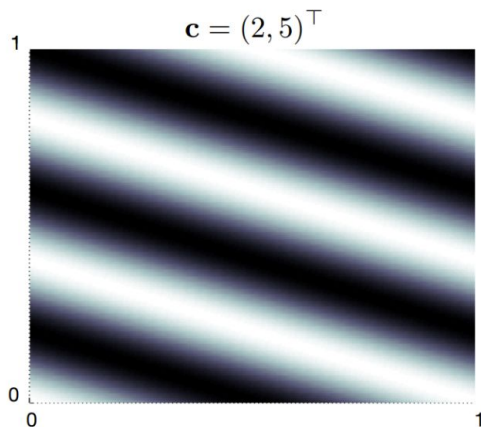
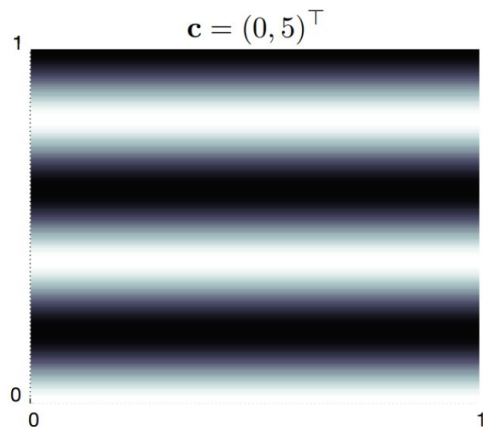
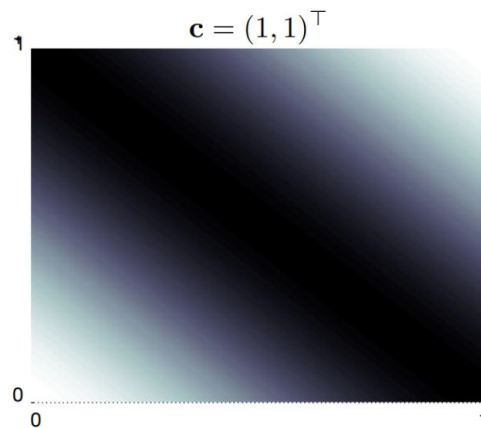
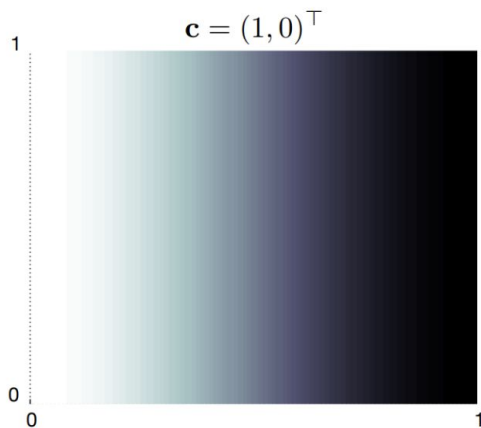
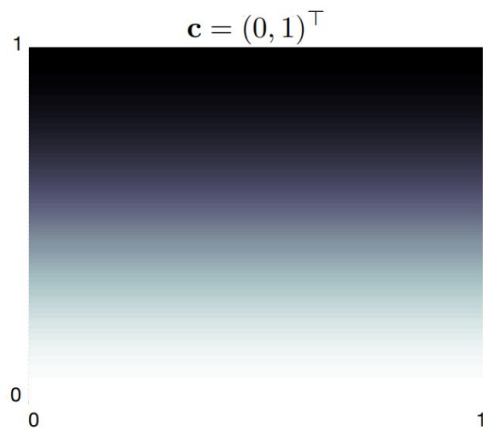
- Fourier:



2D polynomial features

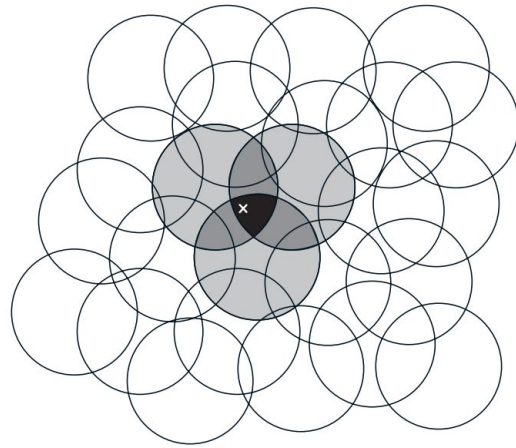


2D Fourier features

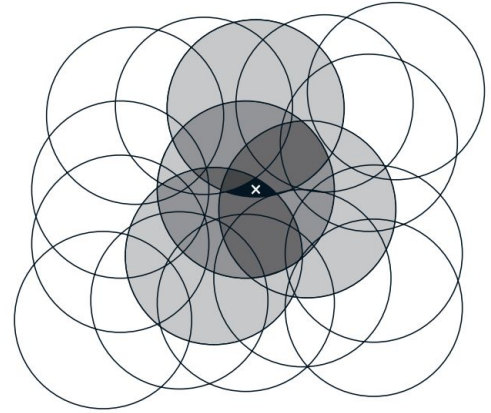


2D Course coding

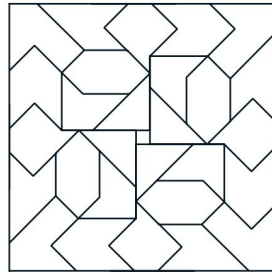
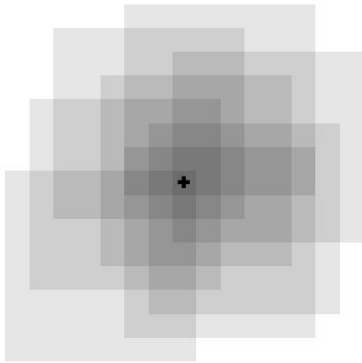
- Each feature is a circle
- If s inside circle, $x_i(s)=1$
- Other shapes possible!



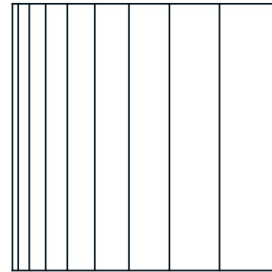
Narrow generalization



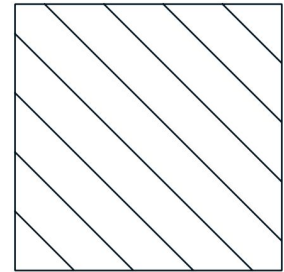
Broad generalization



Irregular



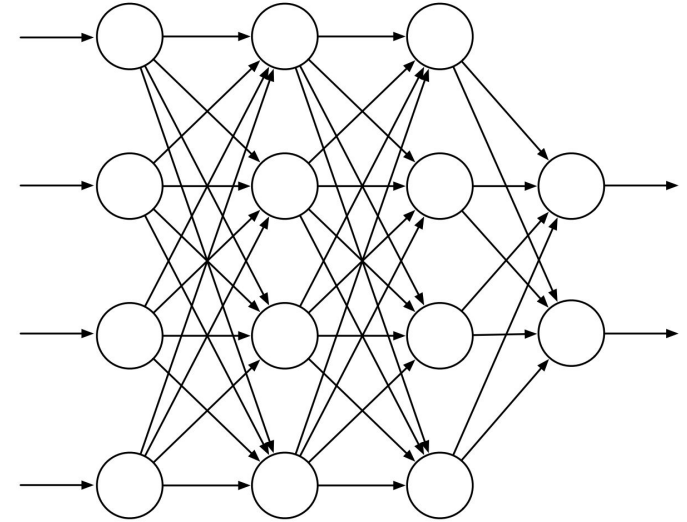
Log stripes



Diagonal stripes

Artificial Neural Networks

- Huge research field by itself!
- In essence, nonlinear function approximation
- Example: multilayer perceptron
- Derivatives easy and fast to compute
- Problem: overfitting

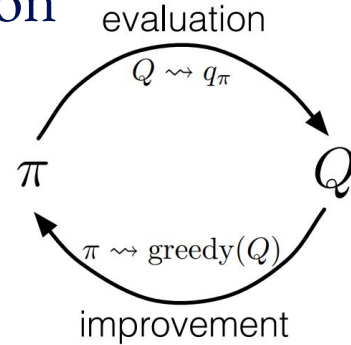


On-policy TD control using approximation

- We know how to estimate state values using approximation
- What if we want to find good policies?

On-policy TD control using approximation

- We know how to estimate state values using approximation
- What if we want to find good policies?
- Use same approach as before:



State-action values with generalized policy iteration!

~~$$v_\pi(s) \approx v(s, w)$$~~

$$q_\pi(s, a) \approx q(s, a, w)$$

On-policy TD control using approximation

State-action values with policy iteration!

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \left[R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t) \right] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$

What would you call this method?

On-policy TD control using approximation

State-action values with policy iteration!

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \left[R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t) \right] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$

SARSA!

On-policy TD control using approximation

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

$S, A \leftarrow$ initial state and action of episode (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 If S' is terminal:

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

 Go to next episode

 Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε -greedy)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

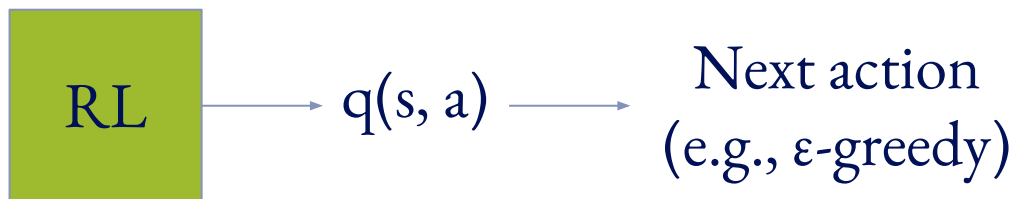
$S \leftarrow S'$

$A \leftarrow A'$

Policy-based RL

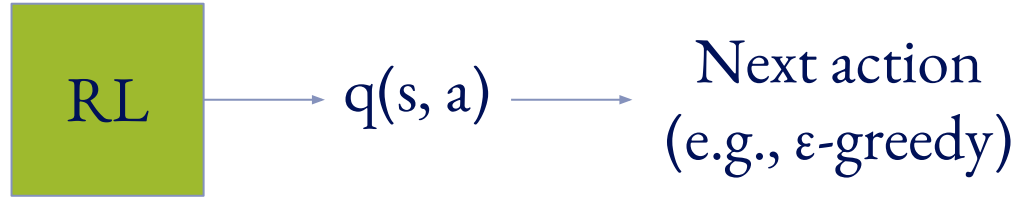
Policy-based RL

Action-value methods:



Policy-based RL

Action-value methods:



Policy gradient methods:



Policy-based RL

- Directly learn a **parameterized policy**
- Policy is a function with θ parameters:

$$\pi(a|s, \theta)$$

“In state s , take action a with probability $\pi(a|s, \theta)$ ”

Policy-based RL

“In state s , take action a with probability $\pi(a|s, \theta)$ ”

Advantages:

- Can approach greedy (i.e. deterministic) policy
- Can create policies with probabilities of selecting actions
- Policies can be easier to determine than values
- Prior knowledge can be injected (e.g. choice of basis functions)

Soft-max in action preferences

- What to use for $\pi(a|s, \theta)$?
- One solution: learn preferences $h(s, a, \theta)$
- Pick actions with high preference with higher probability:

$$\pi(a|s, \theta) \doteq \frac{e^{h(s,a,\theta)}}{\sum_b e^{h(s,b,\theta)}}$$

Preference parametrization

- What to use for $h(s, a, \theta)$?
- Similar to value approximation, many possibilities!

Linear functions, coarse codings, artificial neural networks, ...

- In small problems, can use table of h values

Gradients for policy functions

- Find good θ settings by maximizing performance $J(\theta)$:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t)$$

- What could be a good performance measure?

Gradients for policy functions

- Find good θ settings by maximizing performance $J(\theta)$:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t)$$

- What could be a good performance measure?
- Episodic cases: state value of starting state!

$$J(\theta) \doteq v_{\pi_\theta}(s_0)$$

Gradients for policy functions

- Find good θ settings by maximizing performance $J(\theta)$:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t)$$

- In general, $\nabla J(\theta_t)$ is difficult to compute, but:

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla \pi(a|s, \theta)$$

REINFORCE

- Monte Carlo policy gradient method

Make action more likely

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

Favor actions with high return

Reduce advantage of
likely actions

REINFORCE with baseline

- Possible to show that this is also valid:

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \left(G_t - b(S_t) \right) \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

- Basically, favor actions with return larger than baseline
- Can improve speed of learning

What could be a good baseline?

REINFORCE with baseline

- Possible to show that this is also valid:

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \left(G_t - b(S_t) \right) \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

- Basically, favor actions with return larger than baseline
- Can improve speed of learning

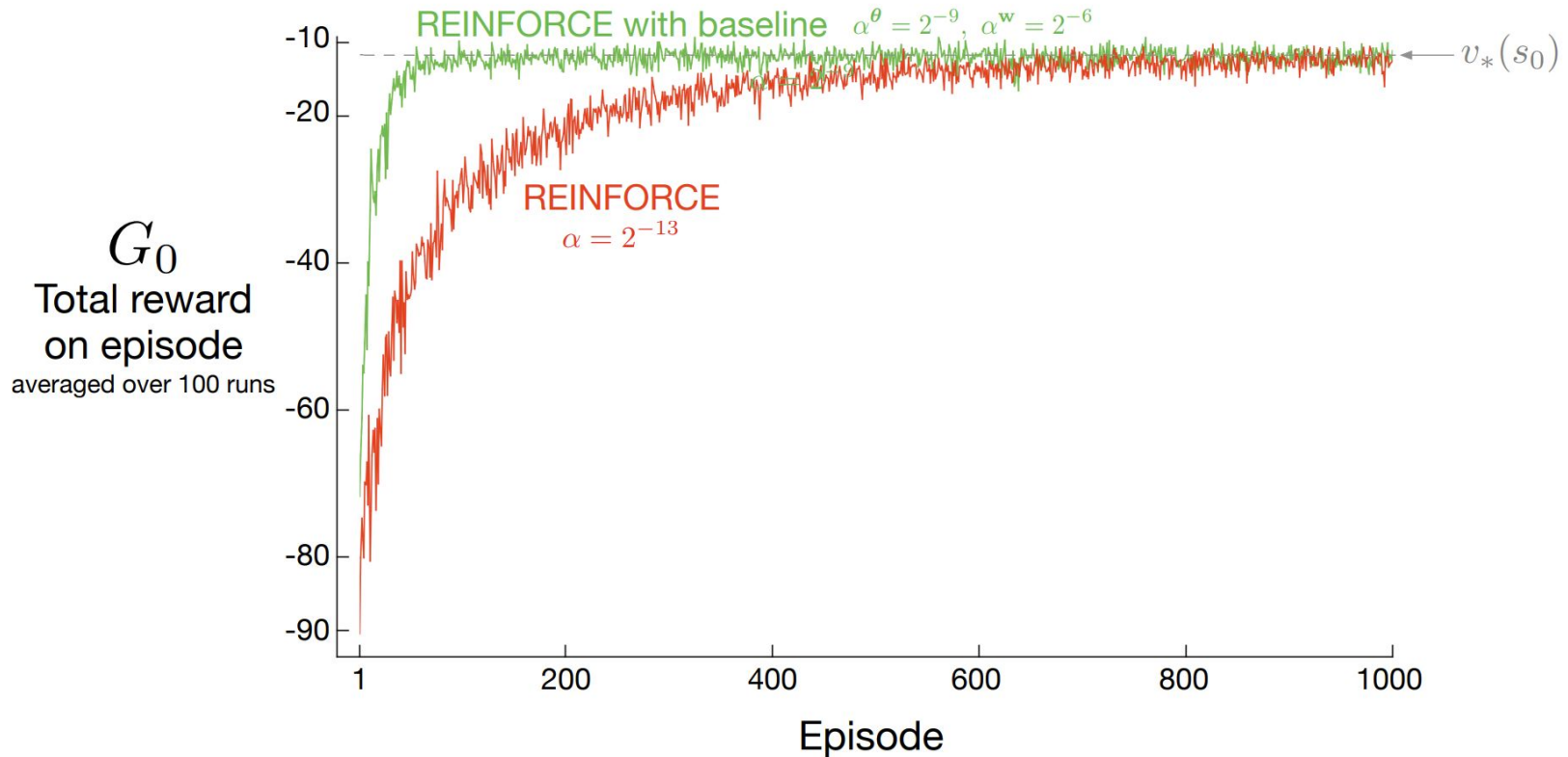
Natural choice: estimate of state value $v(s, w)$

REINFORCE with baseline

- Learn both θ (i.e., policy) and w (i.e., state values) at the same time!
- Usually, natural to use same RL approach for both
- For example, using MC returns:

$$\begin{aligned} G &\leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \\ \delta &\leftarrow G - \hat{v}(S_t, \mathbf{w}) \\ \mathbf{w} &\leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t, \mathbf{w}) \\ \theta &\leftarrow \theta + \alpha^{\theta} \gamma^t \delta \nabla \ln \pi(A_t | S_t, \theta) \end{aligned}$$

REINFORCE with baseline



Actor-critic

- If we are learning state values, can we use them in other ways?
- REINFORCE + baseline (MC):

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \left(G_t - \hat{v}(S_t, \mathbf{w}) \right) \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

Ideas for using $v(s, w)$ in other ways?

Actor-critic

- If we are learning state values, can we use them in other ways?
- REINFORCE + baseline (MC):

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \left(G_t - \hat{v}(S_t, \mathbf{w}) \right) \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

Do temporal-difference learning!

Actor-critic

Do temporal-difference learning!

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \left(G_t - \hat{v}(S_t, \mathbf{w}) \right) \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$



$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \left(R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}) \right) \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

Actor-critic

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \left(R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}) \right) \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

Actor: learns policy

Critic: assess actions

- Natural to pair with single-step TD for learning $v(s, w)$
- Fully online, incremental

Actor-critic

One-step Actor–Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Initialize S (first state of episode)

$I \leftarrow 1$

 Loop while S is not terminal (for each time step):

$A \sim \pi(\cdot|S, \theta)$

 Take action A , observe S', R

$\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$

$\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$

$I \leftarrow \gamma I$

$S \leftarrow S'$

Summary

- Often, huge number of states makes tabular RL impossible
- Solution: approximate with function & learn parameters
 - Many (basis) functions possible!
- Policy-based RL: learn policy instead of action-values
- Use parameterized policy function & learn parameters
- REINFORCE, baselines, actor-critic

Questions?

d.m.pelt@liacs.leidenuniv.nl



Universiteit
Leiden
The Netherlands

